Friedrich-Alexander-Universität
Technische Fakultät

Computer Architecture Practical Exercise

1 Two Dimensional Jacobi

Kenan Gündogan[1] **Philipp Gündisch**[1]

[1]Friedrich-Alexander Universität Erlangen-Nürnberg, Chair of Computer Science 3 (Computer Architecture)

November 7, 2023

2D Jacobi

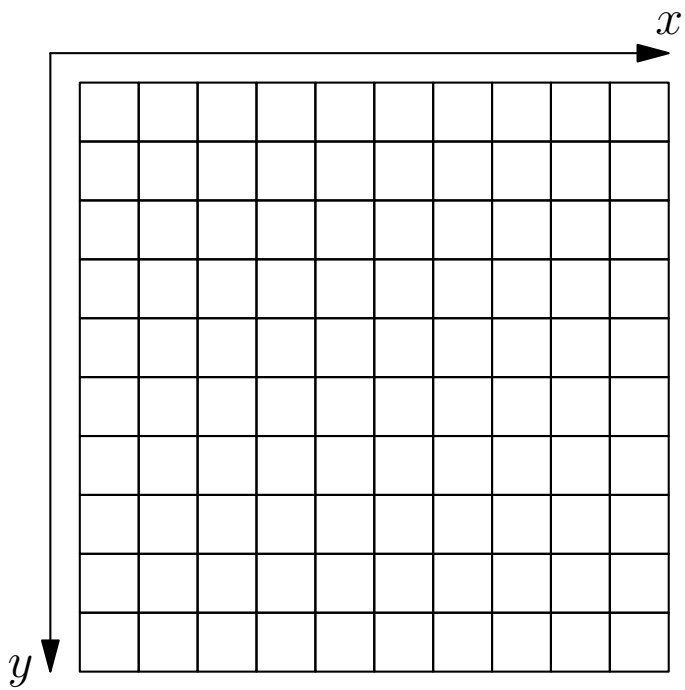
- Simple example for scientific computing
- Used as a numerical solver to process heat propagation
- Naive code is simple
- Code has a huge optimization potential
- Perfect example to illustrate modern CPU features like:
  - SIMD Vectorization
  - Cache Blocking
  - Multi-Core and Many-Core
  - NUMA

Fundamentals

- Jacobi-scheme is a numerical iteration based solver for linear equations of the type $Ax = b$, (e.g. needed to calculate the Δ Laplace operator).
- Laplace operator describes the physics of fields expressed as differential equations:
  - Electrostatic fields
  - Newtons dynamic in fluids and gases
  - Heat propagation
  - etc.

2D Jacobi Heat Propagation

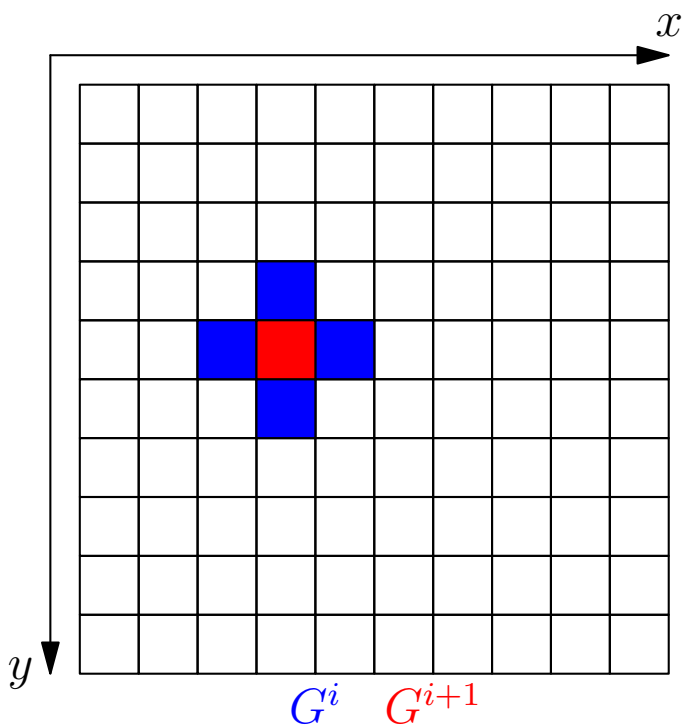
- 2-dimensional plane needs to be discretized.
- Each cell holds the current temperature as a `float` or `double` value.
- Grid is represented as an array of floating point values.



2D Jacobi Heat Propagation

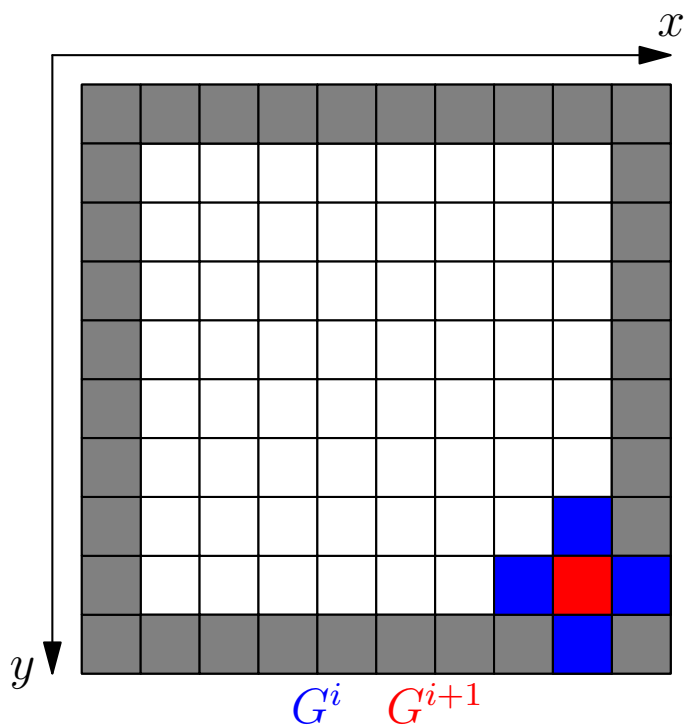
- Processing is time discrete and locally discrete.
- Simulation of one time step t_i to t_{i+1} for coordinates x and y:

$$G^{i+1}_{x,y} = \frac{G^i_{x,y+1} + G^i_{x,y-1} G^i_{x+1,y} + G^i_{x-1,y}}{4}$$



2D Jacobi Update

- For each simulated time step the whole grid needs to be iterated.
- In our example the grid margins remain constant.
- After each iteration the two grids (source and target) can be swapped.

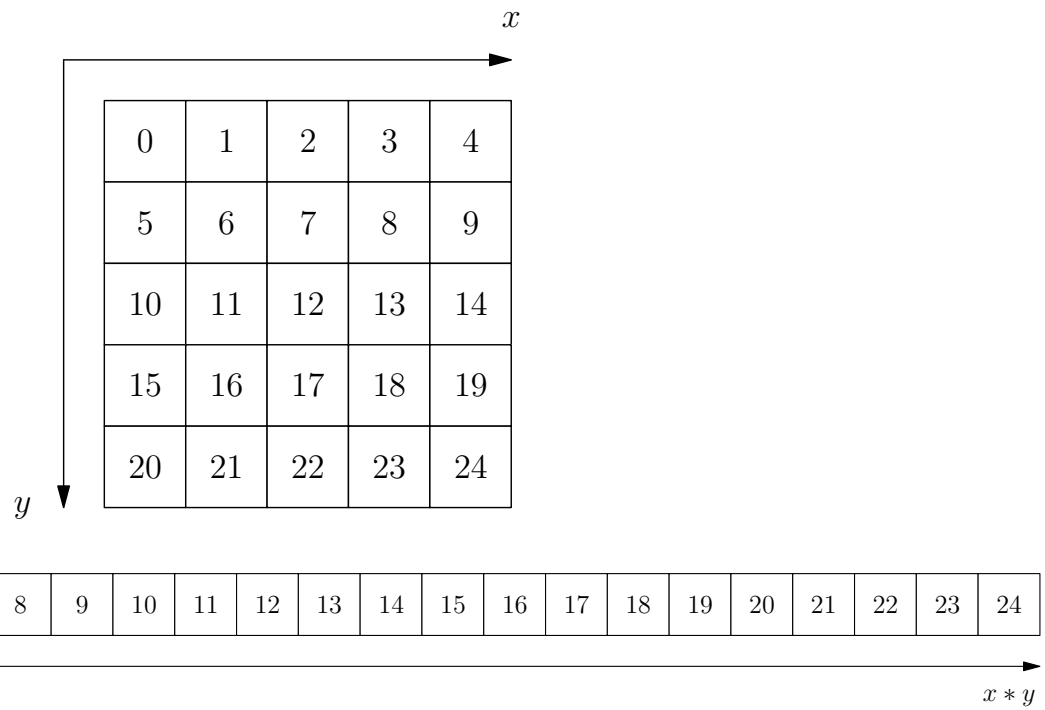


Array Linearization

Use a single memory allocation instead of many:

```
grid1D = malloc(dx * dy * sizeof(double));
```

- A two dimensional array can be accessed with `grid2D[x][y]`.
- A linearized array can be accessed with `grid1D[dx * y + x]`.
- Use the linearized access method!



2D Jacobi

Pseudocode

```
// Iterate over simulation time steps
for(t = 0 ; t < T ; t++) {
    // Update cells
    for(y = 1 ; y < Y-1 ; y++) {
        for(x = 1 ; x < X-1 ; x++) {
            grid_trgt[y*X+x] = grid_src[(y-1)*X+x] + ...
        }
    }
    // Switch the pointers for next iteration
    swap(&grid_trgt, &grid_src);
}
```

Task 1.1: 2D Jacobi

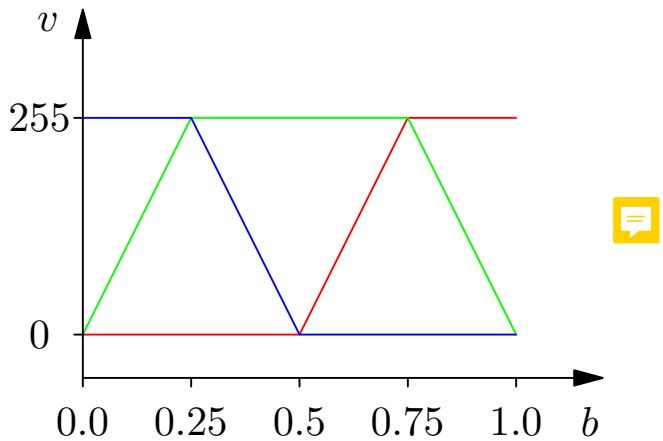
Implementation

- Download new sources and embed it in skeleton of exercise 0.
- Update makefile to build a new target.
- Update the main program to allocate two linearized arrays.
- Use `_mm_malloc()` with alignment on 64 B.
- The new program should receive the grid width and height as parameters.
- Initialize the left and top margin with *1.0*
- Set the remaining cells to *0.0*.
- Implement the *todos* in the new source code.

Task 1.2: 2D Jacobi

Visualization

- The function `draw_grid()` converts the gird to a **Portable Pixmap format** (.ppm) and saves it to a file.
- The header row of the file contains the meta information:
  `P<magic number> <width> <height> <max value>`
  e.g.: P3 128 128 256
- Use P3 or P6 as a format.
- Use the following coloration scheme to map the floating point range $[0.0; 1.0]$ to RGB colors.
- Use this function for debugging purposes during development



Task 1.3: 2D Jacobi

Measurement

- For measurement the number of discrete time steps should be doubled until 1 second is reached.
- Log the same information as in the last exercise and
  - replace *AdditionsPerSecond* with *MegaUpdatesPerSecond* (short MUp/s).
  - add a new column *EdgeSize*
- An update is defined as a single grid cell update.
- $10^6 Up/s = 1 MUp/s$
- Measure from 1kiB to 128MiB (the total memory).
- Use a quadratic array with edge length: $X = Y = \sqrt{\frac{n \cdot 1024B}{2 * 8B}}$

Task Overview

- E 1.1: Jacobi
  - Include new source code
  - Update main.c or create new one
  - Update Makefile
  - Implement jacobi.c
- E 1.2: Visualization
  - Implement draw.c
  - Call draw function from your main.c (for debugging and end result)
- E 1.3: Time Measurement
  - Update the `printf()` format
  - Adapt scripts for measurement
  - Run benchmark on meggie cluster

Appendix: SSH

Login without password prompt

`ssh` and `scp` can be used with **key authentication**. This method is secure and does not require to enter your password each time.

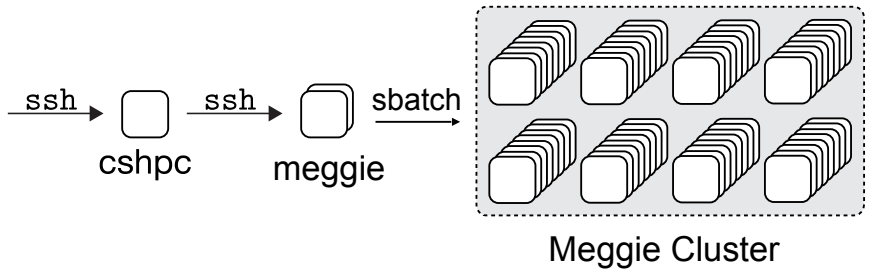
- **ssh key pair generation** (on your computer)
  - `cd ~`
  - `mkdir -p .ssh`
  - `cd .ssh`
  - `ssh-keygen -t ed25519`
  - Give the keypair a meaningful name like 'meggie'
- **Install public key** on computer to log in
  - Copy the `meggie.pub` (public key) file with `scp` to the head node
  - Login to the head node
  - `mkdir -p .ssh`
  - `cat meggie.pub » .ssh/authorized_keys`
  - Delete `meggie.pub`
- **Test login**
  - Log out of head node
  - Login with `ssh -i .ssh/meggie user@host`

This works also for `scp`.

Appendix: SSH

Config File (1/2)

`ssh` and `scp` can be used with an alias name.



- Login to meggie **without** name alias
  - `ssh -i .ssh/meggie user@cshpc.rrze.uni-erlangen.de`
  - `ssh -i .ssh/meggie user@meggie.rrze.fau.de`
- Copy to meggie **without** name alias
  - `scp -i .ssh/meggie ./myfile user@cshpc.rrze.uni-erlangen.de:`
  - `ssh -i .ssh/meggie user@cshpc.rrze.uni-erlangen.de`
  - `scp -i .ssh/meggie ./myfile user@meggie.rrze.fau.de:`
- Login / Copy to meggie **with** name alias
  - *for login:* `ssh meggie`
  - *for copy:* `scp ./myfile meggie:`

Appendix: SSH

Config File (2/2)

To use `ssh` and `scp` with an alias name, a config file needs to be created at `.ssh/config` on your computer.

```
Host cshpc
        HostName cshpc.rrze.uni-erlangen.de
        User <your username>
        IdentityFile ~/.ssh/meggie
        IdentitiesOnly yes
        PreferredAuthentications publickey

Host meggie
        HostName meggie.rrze.fau.de
        User <your username>
        ProxyJump cshpc
        IdentityFile ~/.ssh/meggie
        IdentitiesOnly yes
        PreferredAuthentications publickey
```

Note that the same public key needs to be installed in `.ssh/authorized_keys` of both machines.